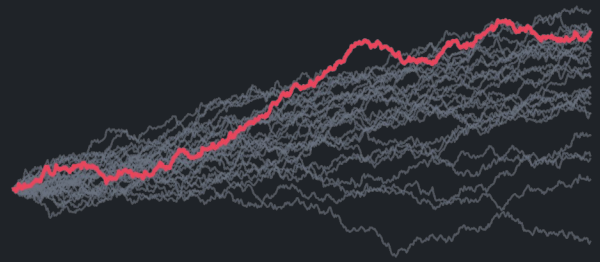
CMSC 838B · STUDENT LECTURE · FALL 2026

Backprop Through Continuous Time

Neural SDEs: modeling, gradients, and training

Running example: when will this avocado go bad?



Nishad Wajge

University of Maryland

RUNNING EXAMPLE

Will This Avocado Be Overripe by Friday?



PLACEHOLDER one fruit, photographed daily (swap in dataset images)

Given the photos so far and the storage temperature c:

$$\mathbb{P}(\tau \leq t^{*} \mid x_{0:k},\ c)$$

τ = day it first reaches stage 4

Public data: 'Hass' Avocado Ripening Photographic Dataset
478 avocados · 14,710 images · photographed daily
Stored at 10 °C, 20 °C, or ambient
5-stage ripening index; stage 4 = end of shelf life
Mendeley Data, 2024 · doi:10.17632/3xd9n945v8

We want a distribution, not a label.
Two fruits that look identical today can spoil days apart: biology and storage add genuine randomness.

Ripeness is a hidden process that drifts and jitters; we only see it through irregular photos.

RUNNING EXAMPLE

What the Model Has to Do

	Image classifier	Sequence model	Latent neural ODE	Latent neural SDE
Irregular, missing photo days	×	*partial*	✓	✓
Uncertainty over future paths	×	*partial*	*partial*	✓
Condition on storage temperature	✓	✓	✓	✓
P(spoiled by day t*)	×	*partial*	*partial*	✓
Physics prior on ripening rate	×	×	✓	✓

Neural SDE = continuous time + real randomness + a drift you can give physical structure.

ROADMAP

Four Parts

1

Continuous time and the adjoint

Neural ODEs; three ways to differentiate a solver

2

Stochastic calculus you need

Itô vs. Stratonovich; a neural SDE for ripening

3

Gradients through randomness

Pathwise vs. score function; the stochastic adjoint; measured costs

4

Training and limits

Latent-SDE ELBO; diffusion models; results; open problems

One question throughout: how do we differentiate through a solver, and what changes with noise?

Continuous Depth: Neural ODEs

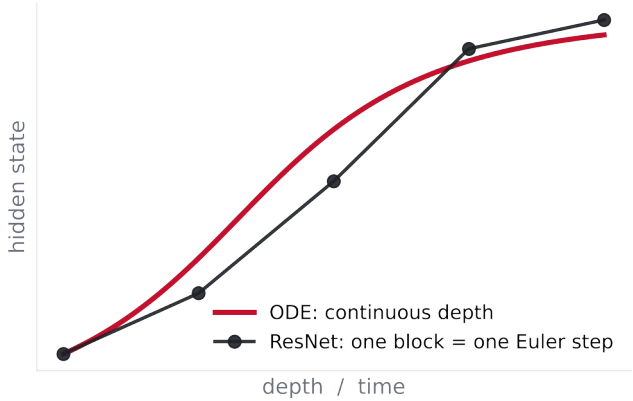
ResNet block

$$h_{k+1} = h_k + f_\theta(h_k)$$

step size → 0

Neural ODE

$$\frac{dh}{dt} = f_\theta(h, t)$$



AVOCADO, DETERMINISTIC VERSION

$$\frac{dz}{dt} = f_\theta(z, t, c), \qquad z(0) = e_\phi(x_0)$$

Encode the first photo, then roll ripeness forward under storage condition c. No randomness after day 0.

A ResNet is Euler's method; a neural ODE hands the stepping to a solver.

Three Ways to Differentiate a Solver

Method	Memory (N steps)	Extra compute	Gradient	Diffrax
Backprop through every step	O(N) stored states	≈ 1 backward pass	exact for the discrete solve	`RecursiveCheckpointAdjoint(checkpoints=N)`
Binomial checkpointing	O(log N) checkpoints	O(N log N) recomputation	exact	`RecursiveCheckpointAdjoint()`
Continuous adjoint	O(1) in N	one backward solve, size 2d + p	approximate: re-solve error	`BacksolveAdjoint()`

d = state size, *p* = number of parameters. Discretize-then-differentiate: rows 1–2. Differentiate-then-discretize: row 3.

Three points on a memory / compute / accuracy frontier.

The Adjoint Method

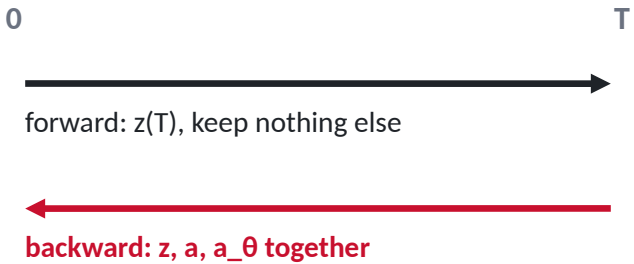
Solve backward from T to 0, all three together:

state, recomputed $$\dot{z} = f_\theta(z, t)$$

adjoint a = ∂L/∂z(t) $$\dot{a} = -a^\top \partial_z f_\theta$$

parameter gradient $$\dot{a}_\theta = -a^\top \partial_\theta f_\theta, \quad a_\theta(T) = 0 \Rightarrow \frac{dL}{d\theta} = a_\theta(0)$$

Start: a(T) = ∂L/∂z(T). Derivation: backup A1.



$a^\top \partial f/\partial z$ and $a^\top \partial f/\partial \theta$ are VJPs

One reverse-mode call of f per solver stage, never the full Jacobian. Same primitive as Lecture 4.

The adjoint is reverse-mode AD in continuous time: each solver stage costs one VJP.

The Itô Toolkit in One Slide

What an SDE means

$$Z_t = Z_0 + \int_0^t \mu(Z_s, s)\, ds + \int_0^t \sigma(Z_s, s)\, dW_s$$

Brownian increments

$$W_t - W_s \sim \mathcal{N}(0,\ (t-s)I), \qquad (dW)^2 = dt$$

Itô's lemma: the extra ½ f″σ² term

$$df(Z_t) = f'(Z_t)\, dZ_t + \tfrac{1}{2} f''(Z_t)\, \sigma^2\, dt$$

Itô → Stratonovich drift

$$\mu_{\text{Strat}} = \mu_{\text{Itô}} - \tfrac{1}{2} \sum_j (\partial_z \sigma_j)\, \sigma_j$$

Euler–Maruyama $Z + \mu\Delta t + \sigma\sqrt{\Delta t}\cdot\varepsilon$: strong order ½, weak order 1. **Milstein**: strong order 1. Full table: backup A4.

Itô is natural for modeling; Stratonovich obeys the ordinary chain rule, which the adjoint needs.

A Neural SDE for Ripening

Photos x_k — irregular days t_k → **Encoder e_φ** — CNN / ViT → **Latent SDE Z_t** — storage c → **Ripeness head** — ordinal, stages 1–5 → **Shelf life τ** — first passage

$$dZ_t = \kappa(c)\,\mu_\theta(Z_t, t)\,dt + \sigma_\theta(Z_t, t, c)\,dW_t$$

drift: neural rate × temperature prior

$$\kappa(c) = \exp\left[-\frac{E_a}{R}\left(\frac{1}{T_c} - \frac{1}{T_{\text{ref}}}\right)\right]$$

Arrhenius scaling: warmer storage → faster ripening.

diffusion: fruit-to-fruit and storage variability

spoilage = first passage of a threshold

$$\tau = \inf\left\{t : q(Z_t) \geq b\right\}, \qquad \mathbb{P}(\tau \leq t^* \mid x_{0:k}, c)$$

PLACEHOLDER

Encoder choice from my project (e.g. compact geometry-aware vision model) goes here

Drift = how fast it ripens; diffusion = how unpredictably; τ = when it crosses stage 4.

Pathwise vs. Score-Function Gradients

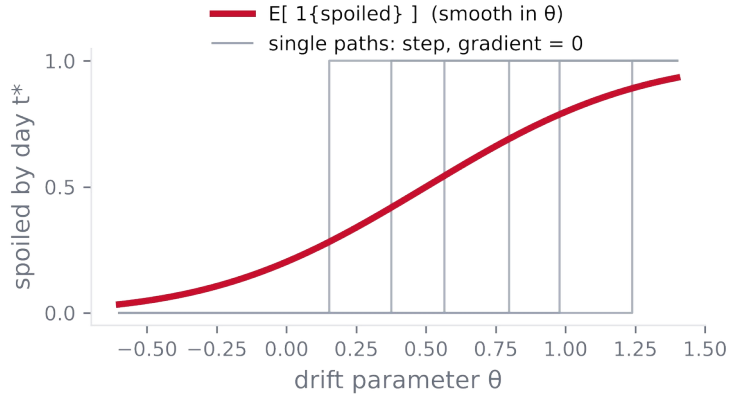
Pathwise (reparameterization): fix W, differentiate the path

$$\nabla_\theta \mathbb{E}[f(Z_T)] = \mathbb{E}[\nabla_\theta f(Z_T(\theta, W))]$$

Score function (likelihood ratio): differentiate the density

$$\nabla_\theta \mathbb{E}[f(Z)] = \mathbb{E}[f(Z)\, \nabla_\theta \log p_\theta(Z)]$$

Pathwise: low variance, needs f differentiable. Score function: works for any f, high variance; for SDEs the density ratio comes from Girsanov.



Smooth surrogate

sigmoid((q − b)/ε): biased, low variance

Brownian-bridge crossing

smooth P(cross) per step (backup A5)

Score function

unbiased, high variance

Pathwise gradients are low-variance but blind to discontinuities, and spoilage is one.

The Stochastic Adjoint

Backward from T, Stratonovich (∘), same Brownian path W:

$$da_t = -a_t^\top \partial_z \mu \, dt \; - \; \sum_j a_t^\top \partial_z \sigma_j \circ dW_t^j$$

$$\frac{dL}{d\theta} = \int_0^T a_t^\top \partial_\theta \mu \, dt + \sum_j \int_0^T a_t^\top \partial_\theta \sigma_j \circ dW_t^j$$

μ = Stratonovich drift (convert Itô first). Li et al. 2020; why: backup A2.

1 Replay the same noise

Store W (memory again), or regenerate W(s) from a seed with a Brownian tree: O(log 1/ε) per query (backup A5).

2 Use Stratonovich

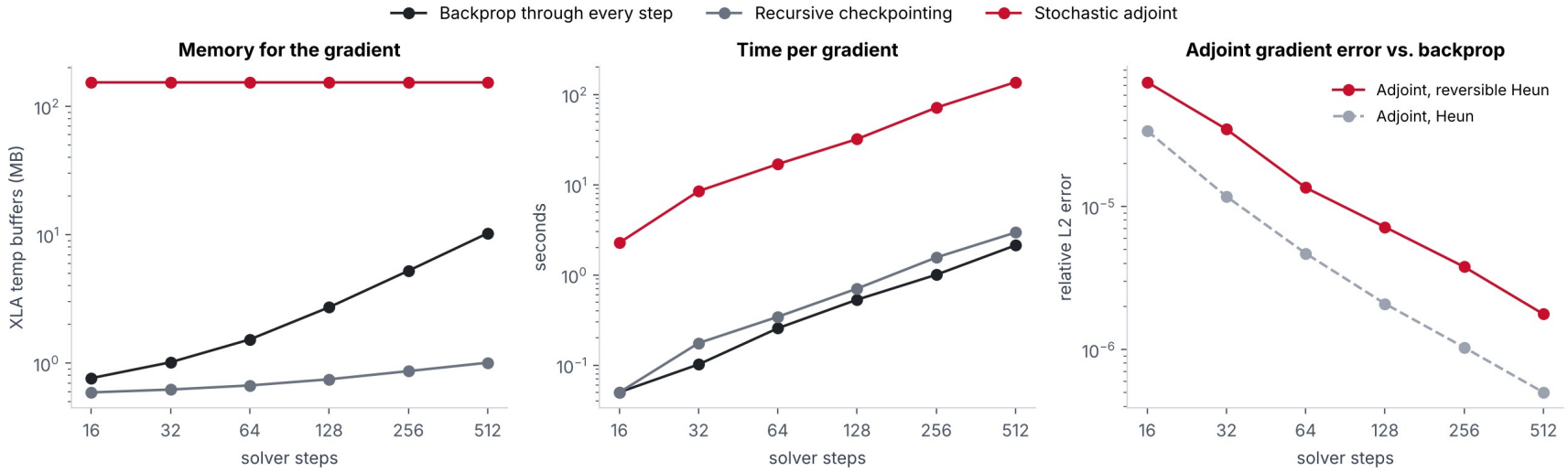
Ordinary chain rule ⇒ the ODE adjoint carries over term by term.

Exact version: reversible Heun is algebraically reversible, so a matching discrete adjoint reconstructs the forward states: gradients exact up to floating-point round-off, at O(1) memory (Kidger et al., NeurIPS 2021; in torchsde). **Diffrax's BacksolveAdjoint solves the continuous adjoint instead, so it stays approximate.**

Same structure as the ODE adjoint, plus a noise term driven by the replayed Brownian path.

PART 3 · MEASURED

Three Ways to Get the Gradient, Measured



16-dim neural SDE (12.7k params), 32 paths, reversible Heun, Diffrax, CPU. At 512 steps: backprop 10.3 MB / 2.1 s · checkpointing 1.0 MB / 3.0 s · adjoint 154 MB / 137 s. Setup: backup A6. Code: github.com/nishadw/neural-sde

In Diffrax the adjoint is flat in memory but large, ~60× slower, and approximate. Checkpointing wins.

Training: the Latent-SDE ELBO

forecast (sees only storage c)

$$\text{prior} \quad dZ_t = h_\theta(Z_t, t, c)\,dt + \sigma(Z_t, t, c)\,dW_t$$

filter (also sees the photos)

$$\text{posterior} \quad dZ_t = h_\phi(Z_t, t, c, x_{0:K})\,dt + \sigma(Z_t, t, c)\,dW_t$$

objective

$$\log p(y_{1:K}) \geq \mathbb{E}_Q\Big[\sum_k \log p(y_k \mid Z_{t_k})\Big] - \mathrm{KL}_0 - \mathbb{E}_Q \int_0^T \tfrac{1}{2}\|u_t\|^2 dt$$

$$\sigma\, u_t = h_\phi - h_\theta$$

y_k = ripening stage of photo k; KL_0 = KL between initial-state distributions.
Derivation: backup A3.

Why a shared σ?

Different diffusions make the two path measures mutually singular: KL = ∞.

Implementation

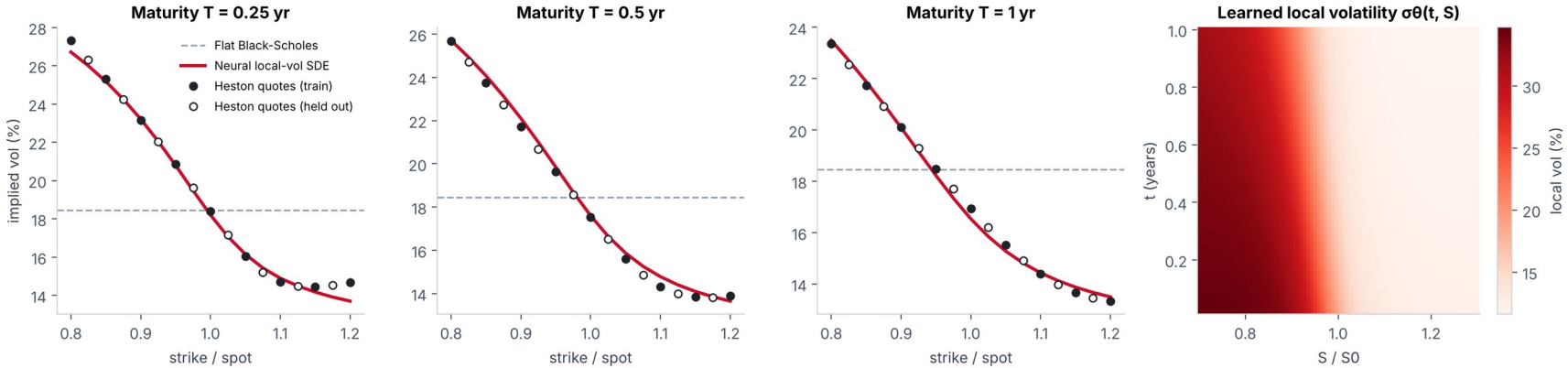
Append ∫½‖u‖² dt as an extra state; each solve gives a Monte Carlo estimate of the KL.

Forecasting

After the last photo, switch to the prior and sample forward to get P(τ ≤ t*).

Girsanov turns the path-space KL into a running cost you integrate alongside the state.

Same Solver, Different Loss



Calibration demo from the repo: neural local-vol SDE fit to a Heston smile by backprop through Monte Carlo. 0.31 vol pts RMSE (0.26 on held-out strikes) vs 4.21 for flat Black–Scholes.

Latent SDE (ELBO)

irregular observations, as in the avocado

SDE-GAN

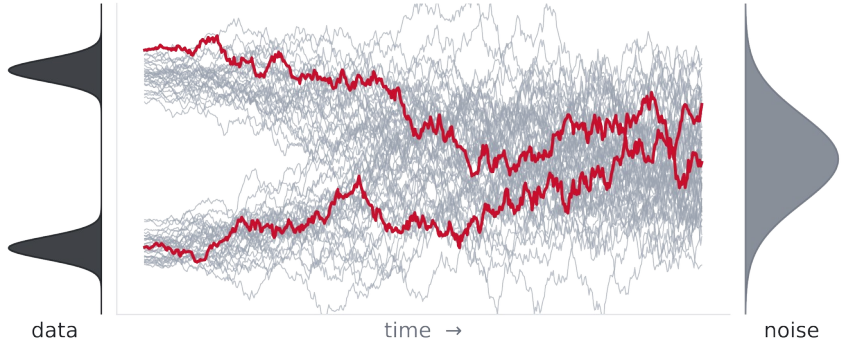
match path distributions; neural CDE discriminator

Calibration

match observed prices or statistics

The differentiable solver is shared; the loss decides what "fit" means.

Diffusion Models Are Neural SDEs



Forward SDE turns data into noise; sampling runs it backward.

vs. our latent SDE: learns μ and σ, trained by backprop through the solver.

forward (fixed)

$$dx = f(x,t)\,dt + g(t)\,dW$$

reverse (learned score)

$$dx = \left[f(x,t) - g(t)^2\, s_\theta(x,t)\right]dt + g(t)\,d\bar{W}$$

probability-flow ODE

$$\frac{dx}{dt} = f(x,t) - \tfrac{1}{2}\, g(t)^2\, s_\theta(x,t)$$

training: denoising score matching

$$\min_\theta \; \mathbb{E}_{t,\,x_0,\,x_t} \left\| s_\theta(x_t,t) - \nabla_{x_t} \log p_t(x_t \mid x_0) \right\|^2$$

A diffusion model is a neural SDE with fixed σ, trained without ever solving it.

RUNNING EXAMPLE

Avocado Shelf Life: Results

PLACEHOLDER

Forecast fan chart

held-out fruit per storage condition: predicted ripeness band vs. observed stages

PLACEHOLDER

Shelf-life calibration

reliability diagram: predicted $P(\tau \le t^*)$ vs. observed frequency

PLACEHOLDER

Baselines × metrics table

classifier · GRU with time gaps + hazard head · latent ODE · latent SDE — C-index, Brier score, NLL

PLACEHOLDER *one-line headline result goes here*

Where Neural SDEs Show Up

Generative models

image, audio, video sampling

Song et al., 2021

Weather ensembles

GenCast: diffusion-sampled forecasts

Price et al., Nature 2025

Robot policies

Diffusion Policy; π0 flow matching

Chi et al. 2023; Black et al. 2024

Quant finance

calibrated, arbitrage-free market models

Gierjatowicz 2022; Cohen 2021

Clinical time series

sparse, irregular vitals and labs

Rubanova 2019; Kidger 2020 (ODE/CDE)

Food supply chain

shelf-life forecasts for inventory and markdowns

this talk

Anywhere the answer is a distribution over trajectories.

Where It Breaks: Open Problems

1	**Adjoint gradients drift**	backward re-solve ≠ forward path; measured ~1e-5 here	*Gholami 2019; slide 12*
2	**Stiffness and step counts**	learned dynamics stiffen; NFE climbs during training	*Finlay et al., 2020*
3	**Discontinuous objectives**	spoilage indicator: pathwise gradient is zero	*slide 10*
4	**Identifiability**	latent scale, drift and diffusion trade off; anchor with the ordinal head	*open*
5	**Visual shift**	lighting or camera changes can look like ripening	*encoder robustness*

Constant memory is real; exact, unbiased, identifiable gradients each take extra work.

DISCUSSION

Three Questions

1. Shelf life is a first-passage time. Smooth surrogate, Brownian-bridge correction, or score function: which would you train with, and why?

2. Diffusion models fix σ; our model learns it. When is learning the noise worth the identifiability trouble?

3. Which differentiable simulator from this course would you make stochastic, and what would happen to its gradients?

Code, benchmarks and slides: github.com/nishadw/neural-sde

Deriving the ODE Adjoint

1. Add the dynamics as a constraint with multiplier a(t)

$$\mathcal{L} = L(z(T)) - \int_0^T a(t)^\top (\dot{z} - f_\theta(z,t))dt$$

2. Differentiate in θ and integrate $a^\intercal$ (d/dt)(∂z/∂θ) by parts

Terms in ∂z/∂θ collect into [∂L/∂z(T) − a(T)]·∂z(T)/∂θ + ∫ (ȧ + $a^\intercal$∂f/∂z)·∂z/∂θ dt

3. Choose a to make both brackets zero

$$\frac{d\mathcal{L}}{d\theta} = \int_0^T a^\top \partial_\theta f \, dt \quad \text{when} \quad \dot{a} = -a^\top \partial_z f, \quad a(T) = \partial_z L$$

The initial-state term ∂z(0)/∂θ is zero when z(0) doesn't depend on θ; otherwise add $a(0)^\intercal$ ∂z(0)/∂θ.

Choose a(t) to cancel every ∂z/∂θ term; what's left is the gradient.

Why Stratonovich for the Stochastic Adjoint

$$dZ_t = \mu\, dt + \sigma \circ dW_t \quad \Longleftrightarrow \quad \dot{Z} = \mu + \sigma\, \dot{W} \;\; (\text{ordinary chain rule})$$

Itô

Left-endpoint integral: a martingale, natural for modeling and for Monte Carlo.

Chain rule gains the ½f″σ² term, and time reversal needs extra corrections.

Stratonovich

Midpoint integral: the ordinary chain rule holds, so dynamics read as an ODE driven by noise (rough-path view).

Convert an Itô model with the drift correction from slide 8, then apply the ODE adjoint term by term.

Stratonovich SDEs behave like ODEs driven by Ẇ, so the ODE adjoint carries over.

Girsanov and the Path-Space KL

Girsanov: change of drift ⇒ density ratio on paths (σu = h_φ − h_θ)

$$\frac{dQ}{dP} = \exp\left(\int_0^T u_t^\top \, dW_t^P - \tfrac{1}{2} \int_0^T \|u_t\|^2 dt \right)$$

Under Q, W^P = W^Q + ∫u dt, and the Itô integral has mean zero:

$$\mathrm{KL}(Q \,\|\, P) = \mathbb{E}_Q\left[\log \tfrac{dQ}{dP}\right] = \mathbb{E}_Q \int_0^T \tfrac{1}{2}\|u_t\|^2 dt$$

Requires: same σσ$^\top$ for prior and posterior; Novikov's condition $E_P \exp(\tfrac{1}{2}\int\|u\|^2 dt) < \infty$ (holds for bounded u).

In code: augment the state with ℓ_t, $d\ell = \tfrac{1}{2}\|u\|^2\,dt$; ℓ_T is a one-sample Monte Carlo estimate of the KL.

Same σ, different drift: the log density ratio is a stochastic integral plus a running cost.

SDE Solvers at a Glance

Solver	Converges to	Strong order	Weak order	Notes
Euler–Maruyama	Itô	½ (1 for additive noise)	1	cheapest; the default
Milstein	Itô or Strat.	1	1	needs $\partial\sigma$; commutative noise in d > 1
Heun	Stratonovich	½ (general noise)	—	predictor–corrector
Reversible Heun	Stratonovich	½ (general noise)	—	algebraically reversible ⇒ exact discrete adjoint
SRA1 / ShARK	Stratonovich	1.5, additive noise only	—	higher order when σ is state-independent

Strong order: pathwise error ~ Δt^order. Weak order: error in E f(Z_T). — = not used here. Sources: Kloeden & Platen (1992); Kidger et al. (2021); Diffrax docs.

Pick the solver for the calculus you wrote, and the order for the quantity you need.

Brownian Bridges: Replay and Crossings

Virtual Brownian tree: sample W(s) given its neighbours, recursively

$$W_s \mid W_u, W_v \sim \mathcal{N}\Big(W_u + \frac{s-u}{v-u}(W_v - W_u),\ \frac{(s-u)(v-s)}{v-u}\Big)$$

Seeds are derived from the interval, so the same query always returns the same W(s): O(1) memory, O(log 1/ε) work.

Crossing probability between solver steps (σ_q = local volatility of q, frozen over the step)

$$\mathbb{P}(\text{cross } b \mid q_k, q_{k+1}) = \exp\Big(-\frac{2(b-q_k)(b-q_{k+1})}{\sigma^2 \Delta t}\Big)$$

Spoilage estimate: $P(\tau \le t^*) \approx 1 - \Pi_k (1 - p_k)$, with $p_k = 1$ if an endpoint is already ≥ b. Smooth in the endpoints, so pathwise gradients work; drift drops out after conditioning on the endpoints. Here $\sigma_q^2 = \|\nabla q(Z_k)^\top \sigma_\theta(Z_k)\|^2$.

One formula does two jobs: regenerate W on demand, and give a smooth crossing probability.

Benchmark Setup (Slide 12)

Model: 16-dim state; drift and diagonal-diffusion MLPs, width 64, depth 2 (12.7k parameters).

Batch: 32 paths solved as one 512-dim SDE with diagonal noise (one adjoint accumulator).

Solver: reversible Heun; Virtual Brownian tree, tolerance 1/4096 (same paths at every resolution); float64; 2-core CPU.

Memory: XLA's compiled temporary buffers for the gradient. Error: relative L2 vs. full backprop.

Adjoint memory scaling (64 steps): 256 → 1,024 noise channels: 62 → 438 MB. 4k → 42k parameters (512 channels): 95 → 389 MB. Backprop ≤ 3 MB throughout. The method needs only 2d + p ≈ 110 KB; the rest is a Diffrax/XLA implementation constant.

Trend is robust; absolute numbers depend on hardware and model size.

References

Chen et al. Neural Ordinary Differential Equations. NeurIPS 2018.

Rubanova, Chen, Duvenaud. Latent ODEs for Irregularly-Sampled Time Series. NeurIPS 2019.

Li, Wong, Chen, Duvenaud. Scalable Gradients for SDEs. AISTATS 2020.

Kidger et al. Neural SDEs as Infinite-Dimensional GANs. ICML 2021.

Kidger, Foster, Li, Lyons. Efficient and Accurate Gradients for Neural SDEs. NeurIPS 2021.

Kidger. On Neural Differential Equations. PhD thesis, 2022.

Mohamed et al. Monte Carlo Gradient Estimation in ML. JMLR 2020.

Gholami, Keutzer, Biros. ANODE. IJCAI 2019.

Finlay et al. How to Train Your Neural ODE. ICML 2020.

Song et al. Score-Based Generative Modeling through SDEs. ICLR 2021.

Lipman et al. Flow Matching for Generative Modeling. ICLR 2023.

Kloeden & Platen. Numerical Solution of SDEs. Springer 1992.

Särkkä & Solin. Applied Stochastic Differential Equations. CUP 2019.

Griewank & Walther. Revolve: binomial checkpointing. ACM TOMS 2000.

Gierjatowicz et al. Robust Pricing and Hedging via Neural SDEs. J. Comp. Finance 2022.

'Hass' Avocado Ripening Photographic Dataset. Mendeley Data, 2024.